Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI5CC · CLOUD COMPUTING · CAPSTONE

Capstone Project

Campus Lost and Found: a scalable cloud application on AWS, from proposal to demo.



Heng Rathpisey · Department GIC · Academic year 2025–2026

What the capstone is

One application, designed, deployed and documented *by your team*.

Apply

Apply all the course concepts in one working system.

Integrate

Integrate multiple AWS services, the way real applications do.

Communicate

Document the architecture and your decisions, then present and demo the result.

Source: course specification

Where it counts

30%

Lab assignments and capstone project

Together they are the largest part of the continuous assessment.

CLO2: implement

Implement scalable cloud applications using AWS services, DevOps practices, Infrastructure as Code and containerization.

CLO3: document

Document architectures, configurations and implementation decisions through professional project documentation.

Source: course specification

Timeline

Week	Stage	What your team does
12	Planning	Project proposal and architecture design: define requirements, create the architecture diagram
13	Development	Begin implementation: implement the core features
14	Implementation	Complete the implementation, test the functionality, optimize the architecture
15	Review	Review session
16	Submission	Final examination and project submission

Work consistently on the project throughout the semester, with an update every week.

Source: course specification

THE PROPOSED PROJECT

Campus Lost and Found

Students report lost and found items with a photo. The cloud stores them, labels them and helps match them.

Capstone

Why this project

A real problem

Everyone on campus has lost something. The users, the data and the success criteria are easy to understand.

Small, but complete

A few screens and one API. Small enough to finish, yet it needs storage, compute, a database, a network and security.

Touches every topic

Each week of the course adds one part of it, from EC2 to an AI service.

What users can do

01

Report an item

A student reports a lost or found item with a title, a place, a date and a photo.

02

Browse and search

Anyone can list recent items and search them by keyword or by label.

03

Get suggested matches

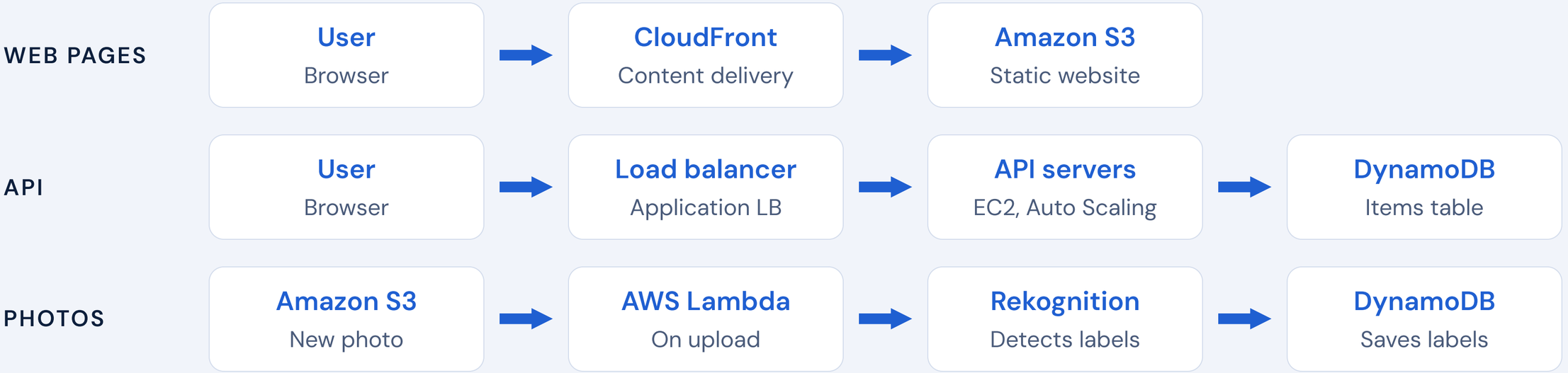
Each photo is labeled automatically, and items with the same labels are suggested as possible matches.

04

Close the case

The reporter marks the item as returned, and it leaves the active list.

Proposed architecture



All of it sits in your own VPC where it applies, is defined with Terraform, and is deployed by a pipeline.

How the course maps to the project (1)

Week	Topic	In the project
3	EC2 and virtualization	The API runs on EC2 instances, packaged as a Docker container
4	Scaling and load balancing	An Auto Scaling group across two zones, behind an Application Load Balancer
5	Cloud storage	Photos and the static website live in Amazon S3, with a lifecycle rule
6	Networking	A custom VPC with public and private subnets, and CloudFront in front
7	Databases	Items and their labels are stored in Amazon DynamoDB

How the course maps to the project (2)

Week	Topic	In the project
9	Serverless	A Lambda function runs on each photo upload event
10	Security and IAM	Least-privilege permissions, security groups, no secrets in the code
11	Cost management	A cost estimate, cost allocation tags, and a budget alert
12	DevOps and IaC	Terraform defines the infrastructure. A CI/CD pipeline builds and deploys
13	AI/ML services	Amazon Rekognition labels each photo

Minimum requirements

1. The application works end to end: report, list, search, close.
2. Photos are stored in Amazon S3 and items in a managed database.
3. At least two instances behind a load balancer, in two Availability Zones.
4. One serverless function triggered by an event.
5. The infrastructure is created from Terraform, not by hand.
6. The code is in Git, with a README.

ASK YOURSELF

The test

Could another team rebuild your system from your repository and your deployment guide alone?

Stretch goals

AI matching

Suggest matches between lost and found items from their Rekognition labels.

CI/CD pipeline

Every push to the main branch builds, tests and deploys automatically.

Auto scaling policy

Show the group scaling out under load and back in afterwards.

CloudFront

Serve the website and the photos through the CDN.

Notifications

Email the reporter when a possible match appears.

Monitoring

A dashboard and an alarm for errors and latency.

Finish the minimum first. A working small system beats an ambitious broken one.

WHAT YOU HAND IN

Deliverables and assessment

The documentation and the presentation are assessed, not only the code.

Deliverables

Source code

The application, the Terraform files and the pipeline, in a Git repository.

README

What it is, how to run it, and who did what.

Architecture diagram

The AWS resources and how they relate.

API documentation

Each endpoint, its inputs and outputs, tested with Postman.

Deployment guide

The steps to build the system from nothing, and to tear it down.

Presentation and demo

A poster or slides, a live demo, and questions.

Source: course specification

How the documentation is assessed

Criteria	Excellent	Needs improvement
Project documentation	Comprehensive README, architecture diagrams, API documentation and deployment guide	Poor or incomplete documentation
Technical communication	Clear explanation of the architecture, the AWS services used, the design rationale and the deployment	Unclear or missing explanations
Presentation	Professional presentation, clear demo, excellent answers to questions	Poor presentation or demo issues

Between the two: good work with minor gaps, then basic work with some sections missing.

Source: course specification

Week 12: the proposal

1. The problem, and who the users are.
2. The features you commit to, and the stretch goals.
3. An architecture diagram.
4. The AWS services, with one line on why each.
5. A cost estimate from the AWS Pricing Calculator.
6. Who in the team does what.

FORMAT

Keep it short

Two or three pages. The diagram and the reasons matter more than the length.

Weekly project updates

Done

What was finished since the last update, with a link or a screenshot.

Next

What the team will finish before the next update.

Blocked

What is in the way, and what help you need.

A team with no project update for more than three weeks automatically gets zero for the poster presentation.

Source: course policy

HOW TO WORK

Working as a team in the cloud

Teams of [team size]. Collaborate professionally, and keep everything in Git.

Capstone

Suggested roles



Infrastructure

VPC, load balancer, Auto Scaling group and Terraform.



Backend

The API, the database tables and the Lambda function.



Frontend

The web pages, and how they call the API.



Quality and documents

Testing, the README, the diagram and the guide.

Roles overlap, and everyone must be able to explain the whole architecture at the demo.

Tools

Tool	Used for
AWS Academy Learner Lab, AWS Console, AWS CLI	Creating and inspecting resources
VS Code and Git	Writing code and sharing it with the team
Docker	Packaging the API as a container
Terraform	Defining the infrastructure as code
Postman and a web browser	Testing the API and the website

Source: course specification

Cost and the Learner Lab



50 USD per account

Monitor your AWS usage to stay within the quota. A load balancer and idle instances cost money every hour.



Destroy when you stop

With Terraform, tear the system down at the end of each session and rebuild it at the next one.



Check what the lab allows

Some services and permissions are restricted in the Learner Lab. Test each service early, and adapt the design if needed.

Common pitfalls

Resources left running

The quota is gone before demo day.

Secrets in Git

Access keys pushed to a repository are compromised. Never commit them.

Built by hand

A system clicked together in the console cannot be rebuilt when the lab session resets.

Documentation last

The README and the diagram written the night before show it. Write them as you build.

Demo day checklist

1. The system is deployed and tested an hour before.
2. Report an item with a photo, live.
3. Show the labels appear, and the item in a search.
4. Show the architecture diagram, and explain one design decision and its trade-off.
5. Show the Terraform files and the repository.
6. Say what it costs to run, and what you would improve.

JUST IN CASE

Have a backup

Record a short video of the working system, in case the lab or the network fails.

ONE LAST THING

Build small, deploy early, document as you go.

A simple system that works, and that you can explain, is the goal.